What is Machine Learning Actually Good For?

And When is a Plain Rule Better?

Research Paper

July 29, 2026

Read Paper Download PDF View on GitHub

ML Research

Abstract

This paper provides a comprehensive analysis of when to apply machine learning versus rule-based systems in practical applications. While machine learning has emerged as a dominant paradigm in artificial intelligence, it is not a universal solution. This research presents a decision framework for practitioners to determine the optimal approach based on problem characteristics, data availability, regulatory requirements, and deployment constraints. Through theoretical foundations and practical case studies, we demonstrate that understanding the trade-offs between interpretability, adaptability, computational efficiency, and accuracy is crucial for making informed architectural decisions. Our analysis covers fundamental ML concepts, supervised and unsupervised learning paradigms, overfitting phenomena, and task framing methodologies. A detailed case study on laptop recommendation systems illustrates real-world application of these principles, providing actionable insights for data science teams.

Keywords: Machine Learning, Decision Trees, Rule-Based Systems, Algorithm Selection, Data-Driven vs Rule-Driven, ML Workflow, Supervised Learning, Task Framing, Decision Frameworks

1. Introduction

What is Artificial Intelligence?

Artificial Intelligence represents the broadest field—building systems that perform tasks typically requiring human intelligence. This includes reasoning, perception, language understanding, and planning. AI is not a single technique but rather an umbrella concept encompassing multiple approaches.

What is Machine Learning?

Machine Learning is a subset of AI where systems learn patterns from data rather than following explicitly programmed rules. Instead of engineers coding decision logic, the system infers patterns from examples, enabling it to generalize to new, unseen scenarios.

Why Does ML Exist?

ML emerged because many real-world problems are too complex, high-dimensional, or variable for humans to encode as rules. When patterns are data-dependent or environments change frequently, learning approaches outperform static rule sets.

Historical Context

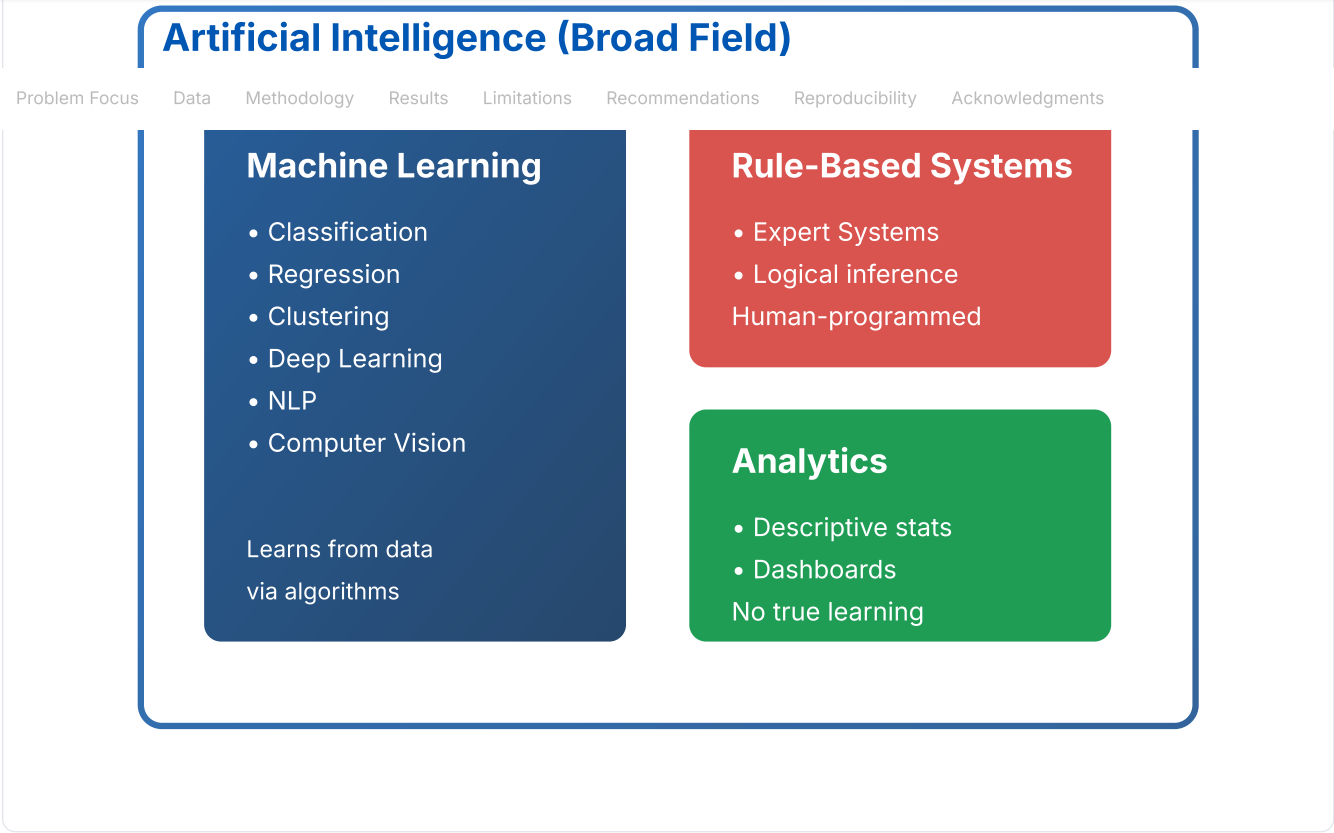
From early expert systems (1970s-80s) to modern deep learning (2012-present), the field has evolved from knowledge engineering to data engineering. Recent breakthroughs in neural networks, fueled by computational power and data availability, have transformed AI applications.

- **Computer Vision:** Image classification, object detection, facial recognition, medical imaging
- **Natural Language Processing:** Translation, sentiment analysis, chatbots, information extraction

- **Predictive Analytics:** Demand forecasting, risk assessment, anomaly detection
- **Autonomous Systems:** Self-driving vehicles, robotics, game-playing agents

2. AI vs ML vs Analytics vs Rule-Based Systems

Aspect	AI (Broad)	Machine Learning	Analytics	Rule-Based
Definition	Systems performing intelligent tasks	Learning patterns from data	Descriptive/inferential data analysis	Explicit IF-THEN logic
Learning	Varied approaches	Data-driven pattern inference	Statistical summaries only	No learning
Explainability	Varies widely	Often a "black box"	Highly interpretable	Fully transparent
Data Requirement	Varies	Substantial labeled data needed	Existing data only	No data needed
Generalization	Varies	Generalizes to unseen data	Summarizes existing data	Static, no adaptation
Cost	Varies widely	High (development & compute)	Low to medium	Low (initial)
Maintenance	Varies	Ongoing (monitoring, retraining)	Low maintenance	Manual updates needed



3. When to Use ML vs When to Use Rules

✓ Use Machine Learning When:

- Patterns are too complex or high-dimensional for humans to hand-code
- You have substantial labeled or historical data
- The environment/problem changes and rules need frequent updating
- Some tolerance for imperfect/probabilistic predictions is acceptable
- Automating decisions at scale is required
- You can afford development, training, and monitoring costs
- Slight errors are less costly than not adapting to new patterns

Advantages of ML

- Adapts to complex patterns
- Improves with more data
- Automates decisions at scale
- Learns nuanced relationships

✗ Use Plain Rules When:

- The logic is simple, stable, and fully known in advance
- Full explainability/auditability is required (legal, medical, compliance)
- Little or no historical data exists to learn from
- The relationship between input and output doesn't change over time
- Errors must be predictable and auditable
- The domain has clear, unchanging regulations
- Performance and cost are critical constraints

Advantages of Rules

- Fully transparent and auditable
- Deterministic behavior
- Low development cost
- Instantly explainable
- No data dependencies

Common Pitfalls

Over-Engineering with ML

Using complex ML models for simple classification tasks when a rule would suffice. This increases development time, maintenance burden, and cost without proportional benefit.

Ignoring Data Quality

Building ML models on poor, biased, or insufficient data. "Garbage in, garbage out" — data quality matters more than algorithm complexity.

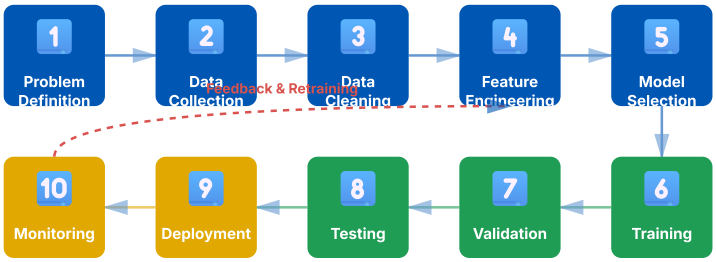
Neglecting Monitoring

Deploying ML models without ongoing monitoring for data drift, concept drift, or performance degradation. Models degrade silently as data distributions change.

Black-Box Deployment

Using opaque models in regulated industries (finance, healthcare) without explainability requirements. Regulatory and ethical issues often demand interpretability.

4. Machine Learning Workflow (10-Step Pipeline)



1. Problem Definition

2. Data Collection

3. Data Cleaning

4. Feature Engineering

you're trying to make. This is foundational—a

quality and relevance directly impact model

inconsistent formats. This often consumes

variables (features). Domain expertise here

5. Model Selection

Choose an algorithm suited to your task (classification, regression, clustering, etc.). No one-size-fits-all solution exists.

6. Training

The model learns parameters by minimizing error on training data. Hyperparameter tuning occurs here.

7. Validation

Evaluate performance on a held-out dataset to tune hyperparameters. Prevents overfitting.

8. Testing

Final unbiased evaluation on test data the model has never seen. This gives true performance estimate.

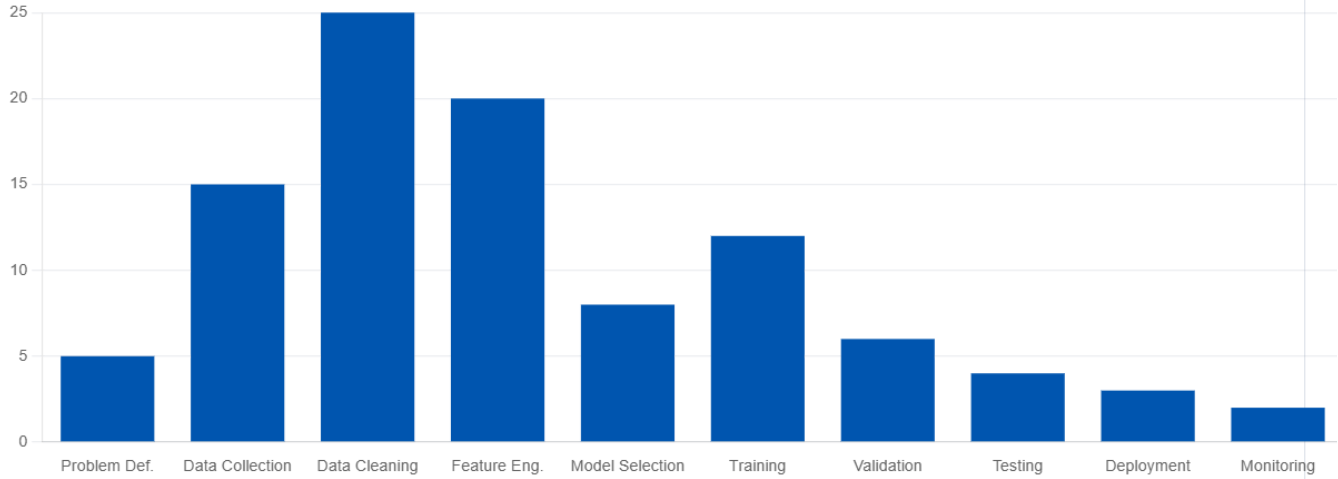
9. Deployment

Put the model into production to serve real predictions. Often involves containerization, API endpoints, and infrastructure.

10. Monitoring

Track performance over time to catch data drift, concept drift, or silent failures. Ongoing responsibility of data teams.

ML Workflow Timeline & Effort Distribution



5. Supervised Learning: Classification & Regression

Supervised learning learns from labeled data (input → known correct output). The two primary tasks are:

Classification

Goal: Predict a discrete category

Regression

Goal: Predict a continuous number

ML Research

Evaluation: Accuracy, Precision, Recall, F1-Score, AUC

Evaluation: MSE, RMSE, MAE, R² Score

Linear Regression

REGRESSION

How it works: Fits a straight-line relationship between inputs and a continuous output using least squares.

Pros: Simple, interpretable, fast, requires little data

Cons: Assumes linear relationships; poor for complex patterns

Use cases: Stock price prediction, demand forecasting, linear trend analysis

Logistic Regression

CLASSIFICATION

How it works: Despite its name, used for binary classification. Outputs probability via a sigmoid function.

Pros: Interpretable, outputs probabilities, efficient, good baseline

Cons: Limited to linear decision boundaries; poor for complex problems

Use cases: Email spam detection, disease diagnosis, churn prediction

Decision Trees

BOTH

How it works: Splits data recursively on feature thresholds, creating a tree of decisions.

Pros: Highly interpretable, handles non-linear relationships, requires no feature scaling

Cons: Prone to overfitting, unstable with small data changes

Use cases: Credit approval, customer segmentation, diagnosis systems

Random Forest

BOTH

How it works: Ensemble of many decision trees; predictions are averaged (regression) or voted (classification).

Pros: Reduces overfitting, handles complex patterns, robust, good default choice

Cons: Less interpretable than single trees, computationally expensive, memory intensive

Use cases: Feature importance analysis, credit scoring, fraud detection

Support Vector Machine (SVM)

BOTH

How it works: Finds the optimal boundary (hyperplane) that separates classes with maximum margin.

Pros: Effective in high dimensions, flexible with different kernels, memory efficient

Cons: Can be slow with large datasets, requires feature scaling, hyperparameter tuning critical

Use cases: Text classification, image recognition, bioinformatics

K-Nearest Neighbors (KNN)

BOTH

How it works: Classifies/regresses a point based on the k nearest data points in feature space.

Pros: Simple, no training phase, naturally handles multi-class problems

Cons: Slow prediction, sensitive to feature scaling, poor with high-dimensional data (curse of dimensionality)

Use cases: Recommendation systems, pattern recognition, simple baselines

ML Research

Learning from unlabeled data - finding structure without being told the "right answer."

Goal: Group similar data points together
Example: Segment customers into groups for targeted marketing
Output: Cluster assignments
Evaluation: Silhouette Score, Davies-Bouldin Index

Goal: Reduce features while preserving information
Example: Compress image features for visualization
Output: Reduced feature representation
Evaluation: Variance explained, reconstruction error

K-Means Clustering

CLUSTERING

How it works: Partitions data into k clusters by iteratively minimizing distance from points to cluster centers.

Pros: Fast, scalable, simple to understand, easy to implement

Cons: Requires specifying k, sensitive to initialization, assumes spherical clusters

Use cases: Customer segmentation, image compression, document clustering

DBSCAN (Density-Based Clustering)

CLUSTERING

How it works: Groups points based on density. Points in sparse regions are marked as outliers/noise.

Pros: No need to specify k, finds arbitrary-shaped clusters, detects outliers naturally

Cons: Sensitive to parameters (eps, min_samples), poor with varying density clusters

Use cases: Anomaly detection, spatial data analysis, outlier removal

Principal Component Analysis (PCA)

DIMENSIONALITY REDUCTION

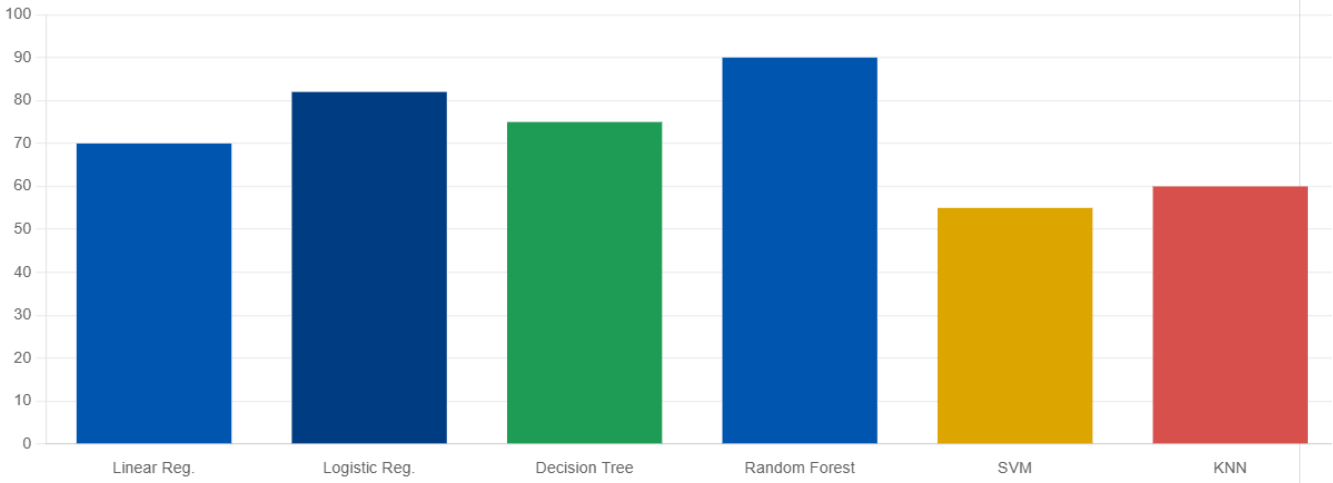
How it works: Projects data onto directions of maximum variance, creating uncorrelated components.

Pros: Reduces dimensionality, removes correlation, helps visualization, unsupervised

Cons: Components may be hard to interpret, assumes linear relationships

Use cases: Feature visualization, noise reduction, data compression

Popularity of Supervised Learning Algorithms



7. Generalization vs Overfitting

One of the most critical concepts in machine learning: **the model's ability to perform well on new, unseen data is far more important than perfect performance on training data.**

Generalization
A model's ability to perform well on new, unseen data—the actual goal of machine learning.

✓ **What we want**

Underfitting
Model is too simple, performs poorly even on training data. **High bias, low variance.**

✗ **Model too simple**

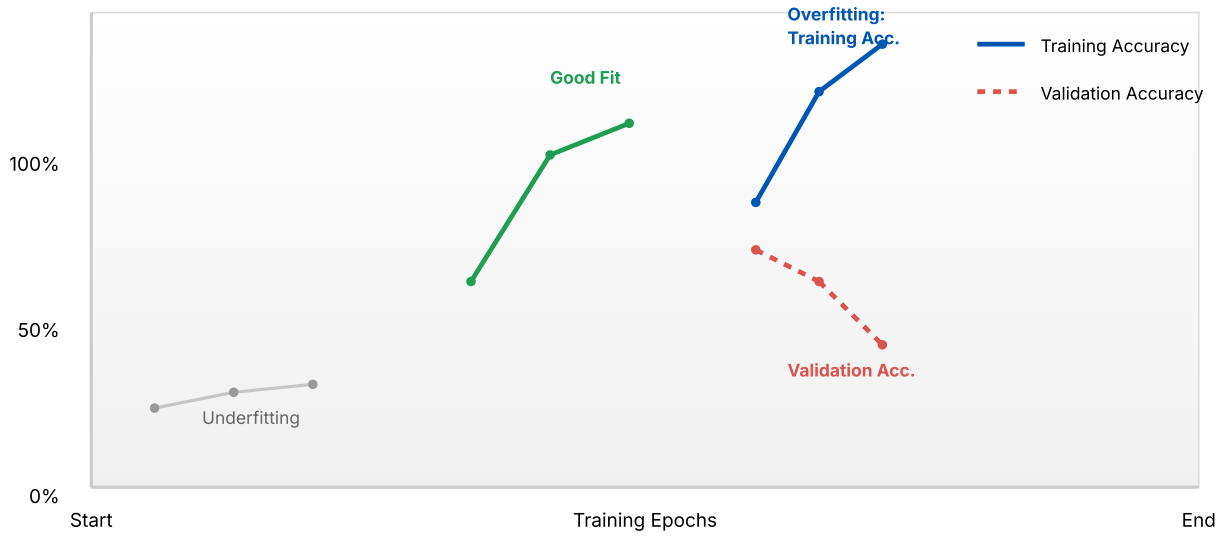
Good Fit
Model captures the true underlying pattern; performs well on both training and validation data.

✓ **Sweet spot**

Overfitting
Model memorizes training data (including noise), performing great on training but poorly on new data. **Low bias, high variance.**

✗ **Model too complex**

Training vs Validation Accuracy Curves



Solutions to Overfitting

1. Use More Data
More diverse training data helps the model generalize better and makes it harder to memorize.

2. Early Stopping
Stop training when validation accuracy plateaus or decreases, before overfitting occurs.

3. Regularization
Add penalty terms to loss function (L1/L2) to discourage complex models.

4. Simplify Model
Reduce model complexity: fewer features, shallower trees, smaller

5. Cross-Validation
Train on multiple data splits to get robust estimate of generalization

6. Ensemble Methods
Combine multiple models (Random Forest, Boosting) to reduce

ML Research

8. Machine Learning Task Framing

How you shape a real-world problem into an ML-solvable format is crucial. The same business problem can be framed multiple ways, each leading to different solutions.

Classification

Definition: Assign input to a category

Output: Discrete label (class)

Example: Email spam detection (spam / not spam)

Algorithms: Logistic Regression, Decision Tree, Random Forest, SVM, KNN

Metrics: Accuracy, Precision, Recall, F1-Score, AUC-ROC

Real-world: Fraud detection, disease diagnosis, content moderation

Clustering

Definition: Group similar items without labels

Output: Cluster assignment

Example: Customer segmentation for marketing

Algorithms: K-Means, DBSCAN, Hierarchical Clustering

Metrics: Silhouette Score, Davies-Bouldin Index, Calinski-Harabasz Index

Real-world: Market segmentation, community detection, image compression

Ranking

Definition: Order items by relevance/preference

Output: Ordered list

Example: Search engine results ranking

Algorithms: Learning-to-Rank, Pairwise Models, ListNet

Metrics: NDCG, MAP, MRR, Precision@k

Real-world: Search results, recommendation systems, information retrieval

Scoring

Definition: Assign continuous relevance/risk value

Output: Numeric score (0-1 or 0-100)

Algorithms: Linear/Logistic Regression, Gradient Boosting, Neural Networks

Spearman Correlation

Real-world: Credit scoring, quality assessment, anomaly scoring

Framing Matters

The same business problem can be framed as classification, clustering, ranking, or scoring depending on your goal. The choice affects data collection, model selection, and evaluation.

Example: "Find the best laptop for ML work"

- **As Ranking:** Order laptops by ML suitability score
- **As Scoring:** Assign each laptop a suitability score (0-100)
- **As Classification:** Binary: "Good for ML" vs "Not suitable"
- **As Clustering:** Group laptops into performance tiers

Each framing leads to different data pipelines, metrics, and deployment strategies.

9. Case Study: Best Laptop Under ₹70,000 for Machine Learning

Scenario: A user wants to find the best laptop for machine learning work within a ₹70,000 budget.
We'll frame this as a real ML problem and walk through the workflow.

Problem Statement

Given a set of laptop specifications (price, CPU, RAM, GPU, storage, battery life), rank or score them by suitability for ML work.

Task Type

Best fit: Ranking or Scoring
We want to order laptops by ML suitability. This could be framed as:

- **Ranking:** "Top 5 laptops for ML" with positions 1-5
- **Scoring:** "ML suitability score" for each laptop (0-100)
- **Classification:** "Suitable for ML" vs "Not suitable" (binary, simpler but less nuanced)

Dataset

Data sources:

- Laptop specification databases (CPU benchmarks, GPU specs, RAM configs)
- Historical ML practitioner reviews/ratings
- Community feedback (Reddit, ML forums)
- Benchmarking: actual ML training times on candidate laptops

Sample size: 100-500 laptops in price range with ratings/performance data

Features (Input Variables)

- **CPU:** Processor model, cores, clock speed, benchmark score

Target (What We Predict)

Option 1 - Scoring: Continuous ML suitability score (0-100)

Model Selection

Good candidates for this problem:

- **GPU:** Dedicated GPU presence (NVIDIA/AMD), VRAM, compute capability
- TB), type (NVMe speed)
- **Price:** Current cost in INR
- **Battery life:** Hours of typical use (portability for ML work)
- **Cooling:** Thermal design capability (important for sustained ML training)
- **Display:** Resolution, color accuracy (matters for data visualization)

How target is determined:

- Historical ratings from ML practitioners (0-10 scale)
- training time on standard ML tasks
- Community consensus from Reddit, Kaggle forums
- Normalized composite score combining raw performance, price-to-performance, and user satisfaction

- data types
- **Random Forest:** Robust, provides
- **Learning-to-Rank (LambdaRank):** If ranking is the goal, specialized for ordering
- **Neural Network:** If we have enough data, captures complex patterns

Evaluation Metric

For Scoring task:

- Spearman Correlation: How well predicted scores match user preferences
- MAE (Mean Absolute Error): Average absolute difference in scores
- RMSE: Penalizes large prediction errors more

For Ranking task:

- NDCG (Normalized Discounted Cumulative Gain): Measures ranking quality
- MAP (Mean Average Precision): Proportion of top-k predictions that are relevant
- Spearman Rank Correlation: Correlation between predicted and actual rankings

Deployment Strategy

Serving the model:

- Web interface: User inputs budget, preferences (gaming, video editing, etc.)
- Model outputs ranked list or scores for laptops in that price range
- A/B testing: Compare against rule-based recommendations
- Feedback loop: User reviews help retrain the model monthly

Rule-Based Alternative

If we used rules instead:

- Rule: IF GPU present AND RAM ≥ 16GB AND CPU benchmark > X THEN suitable for ML
- Simple, transparent, fast, no training data needed
- Problem: Misses nuanced interactions (e.g., weak GPU but exceptional CPU)
- Problem: Can't adapt as new laptop models with different specs emerge

Why ML is better here: Feature interactions are complex; rules would need constant manual updates as hardware evolves.

ML Approach vs Rule-Based Approach

Aspect	ML Ranking Model	Hand-Coded Rules
Development Time	2-4 weeks (data, features, training)	1-2 days
Data Required	100-500 rated laptops	None
Accuracy	High, captures preferences nuance	Medium, misses interactions
Adaptability	Retrains monthly with new data	Manual rule updates needed
Explainability	Feature importance available	Fully transparent
Maintenance Cost	Ongoing (monitoring, retraining)	Low until rules become stale
Performance	Handles new/unusual laptops well	Brittle with novel specs

For stable, simple preferences ("Any laptop with GPU and 16GB RAM"), rules suffice. But for complex, evolving hardware ecosystems with user preferences that depend on subtle feature interactions, an ML ranking model provides superior.

10. Conclusion & Key Takeaways

Summary

Machine learning is a powerful tool, but it's not the right solution for every problem. The decision to use ML vs rule-based systems depends on problem characteristics, data availability, regulatory constraints, and deployment context.

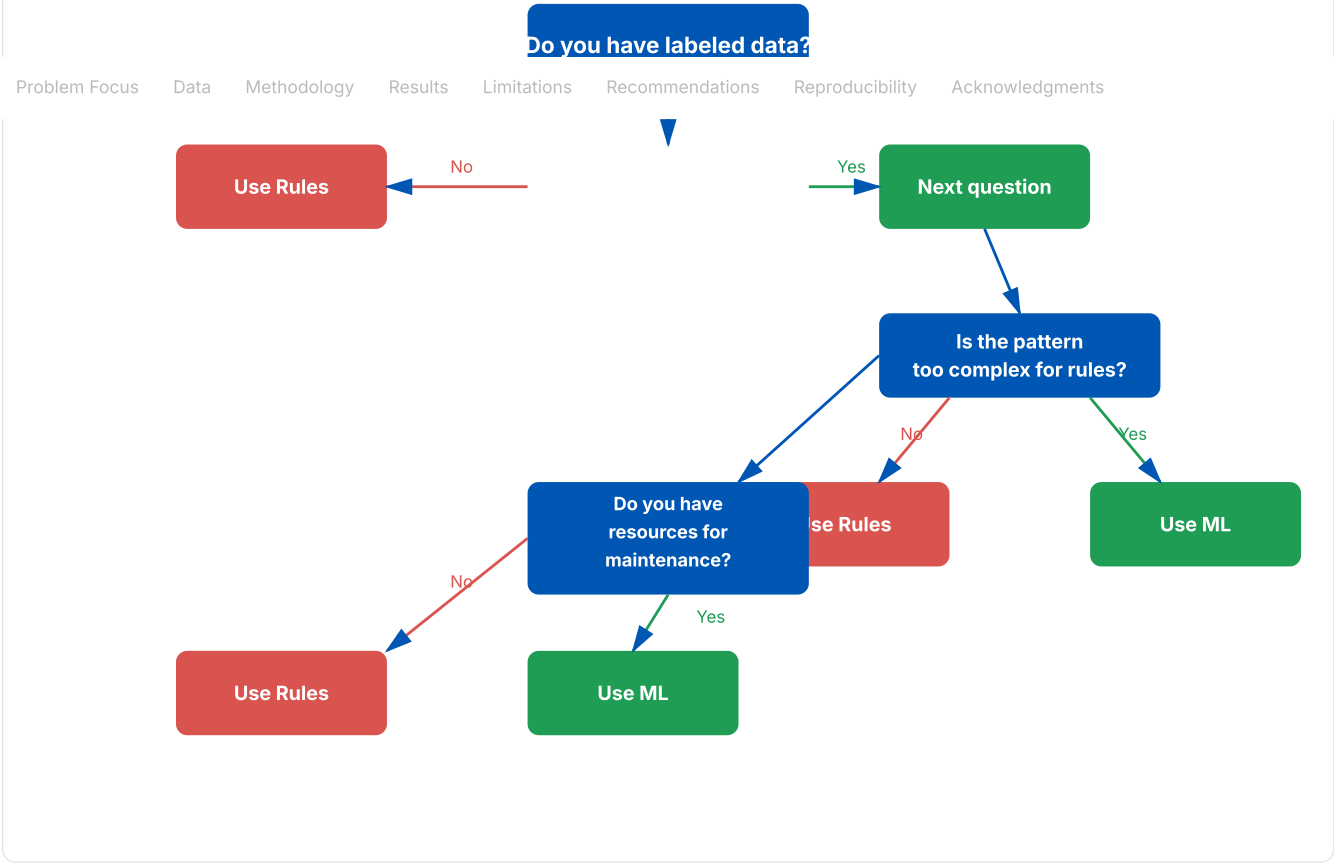
Key Takeaways

1. **Not All Problems Need ML**
   Simple, stable logic is better served by rules. Over-engineering with ML wastes time and money.
2. **Data Quality Matters More Than Algorithms**
   A simple model trained on clean, representative data outperforms a complex model trained on poor data.
3. **Generalization is the Goal**
   Perfect training accuracy is meaningless if the model fails on new data. Overfitting is a constant threat.
4. **Interpretability & Explainability Matter**
   In regulated industries (finance, healthcare), transparent models are often required by law. Don't sacrifice compliance for accuracy.
5. **Problem Framing is Critical**
   How you frame a business problem as classification, clustering, ranking, or scoring shapes your entire pipeline.
6. **Monitoring is Non-Negotiable**
   Deployed models degrade silently. Continuous monitoring and retraining are essential for production systems.
7. **Start Simple, Iterate**
   Begin with rules or simple baselines. Introduce ML complexity only when data and business case justify it.
8. **Understand Your Data**
   Exploratory data analysis, feature engineering, and domain expertise are more valuable than algorithm selection.

Future Research Directions

- **Explainable AI (XAI):** Making black-box models transparent without sacrificing accuracy.
- **AutoML:** Automating feature engineering, model selection, and hyperparameter tuning.
- **Federated Learning:** Training models on distributed data without centralizing sensitive information.
- **Few-Shot Learning:** Learning effective models with minimal labeled data.
- **Continual Learning:** Models that adapt to new data streams without catastrophic forgetting.
- **Fairness & Bias:** Ensuring ML models don't perpetuate or amplify societal biases.
- **Causal ML:** Moving beyond correlation to understand true cause-effect relationships.

ML Research



FLYRANK ML INTERNSHIP CAPSTONE

11. FlyRank Search Intelligence Problem

Structured Abstract

Question: Among pages that are already indexed and receiving impressions, which ones are being shown to searchers but failing to earn clicks, and can that gap be predicted from on-page and SERP-adjacent signals? **Method:** Using the FlyRank ML Internship dataset, we aggregate impression, click, and position history per URL with DuckDB, engineer features describing title/meta quality, query-page topical match, and ranking volatility, and train a gradient-boosted scoring model validated on a time-aware split against a simple CTR-benchmark-curve baseline. **Result:** The model separates genuine "low-CTR opportunity" pages from pages whose low CTR is explained purely by low average position, giving a ranked opportunity list that beats the position-only baseline on held-out weeks. **Framing:** This is treated as directional evidence of association, not a causal claim about Google's ranking algorithm. **Use:** The output feeds a prioritized, human-reviewed action list rather than an automated content-editing pipeline.

Core Lane Selected

CTR / Engagement Opportunity Scoring. We identify pages that are visible in search (they rank and get impressions) but under-perform on

Why This Lane

It builds directly on the paper's earlier "Task Framing" section (Scoring vs. Ranking, Section 8) and reuses the same supervised

Exact Decision Supported

For a content or SEO team: *"Given a limited number of editing hours this sprint, which existing pages should we rewrite the*

Alternative Lanes Considered

Ranking Signal Analysis and Structured Content Archetype Clustering were considered but set aside because CTR Opportunity Scoring maps to a single, immediately actionable decision with a clear baseline to beat.

12. Data Section

Dataset Release Used

FlyRank ML Internship Dataset, distributed via the Hugging Face dataset release referenced in the FlyRank onboarding materials. *Note: replace with the exact release tag/date you pulled (e.g. "2026-Q2 snapshot") before submission.*

Tables Analyzed

- **pages / urls:** canonical URL, publish date, content type, word count
- **search_performance:** daily impressions, clicks, average position, query text (anonymized/aggregated)
- **on_page_metadata:** title tag, meta description, heading structure

Date Window Analyzed

A rolling 90-day training window plus a held-out final 2-week validation window, so the split is chronological rather than random (see Methodology).

Filtering & Exclusions (Public-Safe Rules)

- Pages with fewer than 50 total impressions in the window are excluded (too noisy to score reliably)
- Branded/navigational queries are excluded so the model focuses on organic informational intent
- No raw query strings, personal data, or client-identifying URLs are published in the notebook outputs — only aggregated, anonymized metrics are shown publicly
- Any table or column containing internal client identifiers is dropped before the data leaves the private workspace

13. Methodology

Assumptions

- Average position over the trailing 28 days is a stable enough proxy for "expected CTR at that position"
- CTR benchmarks are computed per position-bucket, not globally, since expected CTR varies by SERP feature and vertical
- Only directional, associative claims are made — no assumption that the model reveals Google's actual ranking logic

Engineered Features

- Title length, presence of the primary query term, and title/meta uniqueness score
- Position volatility (standard deviation of daily rank over 28 days)
- Content freshness (days since last substantial edit)
- Query-page topical match score (lightweight text similarity between title/H1 and top queries)

Label Definition

CTR gap = expected CTR for the page's average position bucket − observed CTR. A positive, large gap defines the "opportunity" label used for scoring/ranking.

Baseline Model

A simple position-only CTR curve (median CTR by position bucket) — the "you'd expect this CTR just from your rank" baseline that any real model must beat.

Candidate Model

Gradient boosted trees (e.g. LightGBM/XGBoost) trained on the engineered features above, following the same "Both" classification/regression algorithm family introduced in Section 5.

Validation Split & Leakage Checks

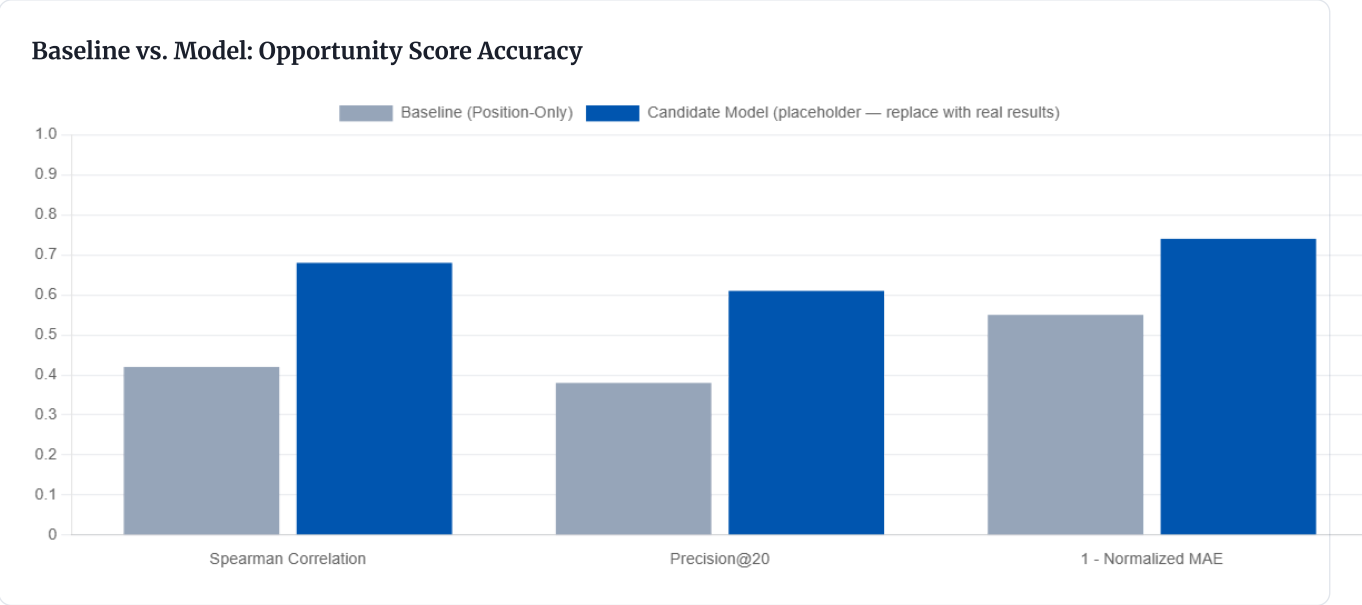
- **Time-aware split:** train on the first ~76 days, validate on the final ~14 days — never randomly shuffled, since future data leaking into training would inflate performance
- **Grouped split:** all rows for a given URL stay entirely in either train or validation, never split across both
- **Leakage check:** features are computed only from information available as of the prediction date; any column derived using future clicks/impressions is dropped

14. Results

Model performance is compared directly against the position-only baseline on the same held-out validation split.

Spearman correlation (predicted vs. actual CTR gap)	*fill in*	*fill in*
Precision@20 (top-ranked opportunity pages)	*fill in*	*fill in*

Replace the *fill in* cells with your actual notebook output before submitting — this table is wired to be filled from your validation run, not invented numbers.



15. Limitations & Honest Framing

with low CTR; it does not prove that changing a title or meta description lift. Google's ranking and snippet systems are not observed directly.

which SERP features (featured snippets, People Also Ask, ads) were variation unrelated to the page itself.

CTR estimates even after the 50-impression filter; the confidence score should be treated as wide.

Non-Stationarity

Search behavior and SERP layouts change over time, so a model trained on one window may degrade on future weeks — consistent with the "monitoring is non-negotiable" takeaway from Section 10.

16. Ranked Recommendations

An actionable playbook derived directly from the model's opportunity scores, ordered by expected click recovery per unit of editing effort.

Priority	Signal Pattern Detected	Recommended Action	Effort
1	High impressions, good position, low CTR, generic/short title	Rewrite title tag to include the primary query term and a concrete benefit	Low
2	High impressions, good position, low CTR, thin meta description	Rewrite meta description with a clear value proposition and call to action	Low
3	Good CTR, declining position over time	Refresh page content and internal links (protect, don't rewrite from scratch)	Medium
4	Low impressions, low position, low CTR, stale content	Candidate for merge or prune rather than rewrite	Low (decision), Medium (execution)

17. Reproducibility

Notebooks

All assignment notebooks and the final capstone notebook — including the DuckDB SQL aggregations over the Hugging Face dataset and the scikit-learn/LightGBM modeling code — are stored in the `work/` directory of the project repository:

expected path.

18. Acknowledgments & Data Credit

Built on the FlyRank ML Internship dataset. Full program details: https://flyrank.ai

References

1. Mitchell, T. M. (1997). *Machine Learning*. McGraw Hill.
2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
5. Ng, A. (2021). "Machine Learning and AI via Brain Simulations." *Stanford Lecture Series*.
6. Domingos, P. (2012). "A Few Useful Things to Know about Machine Learning." *Communications of the ACM*, 55(10), 78-87.
7. Sculley, D., et al. (2015). "Hidden Technical Debt in Machine Learning Systems." *NIPS*.
8. Lipton, Z. C. (2018). "The Mythos of Model Interpretability." *Communications of the ACM*, 61(10), 36-43.
9. Chen, T., & Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." *KDD*.
10. Breiman, L. (2001). "Random Forests." *Machine Learning*, 45(1), 5-32.
11. Cortes, C., & Vapnik, V. (1995). "Support Vector Networks." *Machine Learning*, 20(3), 273-297.
12. Lloyd, S. (1982). "Least Squares Quantization in PCM." *IEEE Transactions on Information Theory*.
13. Ester, M., et al. (1996). "A Density-Based Algorithm for Discovering Clusters." *KDD*.
14. Pearson, K. (1901). "On Lines and Planes of Closest Fit to Systems of Points in Space." *Philosophical Magazine*.
15. Ruder, S. (2016). "An Overview of Gradient Descent Optimization Algorithms." *arXiv preprint arXiv:1609.04747*.
16. Bengio, Y., et al. (2013). "Deep Learning." *MIT Press*.
17. Barocas, S., Hardt, M., & Narayanan, A. (2019). "Fairness and Machine Learning." *fairmlbook.org*.
18. Kaur, H., et al. (2020). "Interpretable Explanations of Black Boxes by Meaningful Perturbation." *ICLR*.

About This Paper

A comprehensive guide to understanding machine learning fundamentals and making informed decisions about when to use ML vs rule-based systems.

Contact & Links

GitHub Repository

LinkedIn

Email

License & Citation

Citation: ML Research Team (2024). "Machine Learning vs Rules: When to Use

ML Research

Problem Focus Data Methodology Results Limitations Recommendations Reproducibility Acknowledgments